National University of Computer and Emerging Sciences, Lahore Campus

	Course:	**Data Science**	**Course Code:**	**CS4048**
	Program:	**BCS – 7th Semester**	**Semester:**	**7**
	Due Date:	**12 September 2026**	**Total Marks:**	**40 (+5 Bonus)**
	Section:	**All BSCS Sections (7A + 7B)**	**Page(s):**	
	Type:	**Assignment 1 (Version A)**	**Topic:**	**Web Scraping**

Which Version Should You Attempt?

This assignment has three versions — **A**, **B**, and **C**. To find out which one is assigned to you, take the **last four digits of your university roll number**, and compute:

```
(last four digits of your roll number mod 3) + 1
```

Then match your result below:

- Result = **1** → attempt **Version A**
- Result = **2** → attempt **Version B**
- Result = **3** → attempt **Version C**

Example: roll number 22K-1234 → last four digits = **1234** → 1234 mod 3 = **1** → 1 + 1 = **2** → this student must attempt **Version B**, and should ignore Version A and Version C.

Important Instructions:

1. Zip your Jupyter Notebook(s), your final dataset CSV file(s), and your short methodology report into a single folder and upload it to the Google Classroom submission link. Name the zip file with your roll number, e.g., **Assignment1_23L_XXXX.zip**. Submissions in any other format will not be accepted and will be awarded zero marks.
2. Use the formula above to work out which version (A, B, or C) is assigned to your roll number, and complete **only that version**. This document is **Version A** — if the formula gives you a different letter, stop, download the correct version, and use that one instead. Submitting a solution to a version other than the one assigned to your roll number will not be accepted.
3. You must submit your own, independent implementation. Solutions will be checked for plagiarism against other students' submissions; if copying is detected, negative marks will be given to every student involved.
4. You are expected to fully understand the code you submit. A short viva may be conducted, and you must be able to explain any part of your scraper, including how you identified elements and handled pagination or dynamic content.
5. Your final dataset must be produced **programmatically** by running your scraper. Manually typing, copying, or editing scraped values into the CSV file is not allowed and will be treated as a violation of the assignment's purpose.
6. Do not attempt to bypass CAPTCHAs, logins, paywalls, or any anti-bot or security mechanism on any website. If a website you choose to explore appears to require this, do not use it — the assigned target URLs below do not require it.

7. Your submitted notebook/script must be able to reproduce your dataset-collection process when re-run (allow for the fact that live websites can occasionally be slow — add reasonable waits/retries rather than hard-coded sleeps only).
8. You must publish your final scraped dataset on **Kaggle or GitHub** and include the public URL in your submission (see Dataset Publication below).
9. Late submission of your solution is not allowed. For each day after the deadline, 20% of the marks will be deducted. Three days after the deadline, no submission will be accepted.

Web Scraping — Version A

Domain: *GameVault Catalog Migration & StyleHub Storefront Prototype*

Background Scenario

StyleHub, an online apparel retailer, has just **acquired GameVault**, a dedicated video-game storefront, and needs a full catalog snapshot from GameVault before folding it into its own systems. Separately, StyleHub is testing a new landing-page prototype for its own catalog that continuously loads more products as a visitor scrolls, rather than through numbered pages.

You are working as a **Data Science Intern** at StyleHub. Your task is to build a scraper for GameVault's full product catalog and a second scraper for StyleHub's own scrolling prototype page, and to turn what they collect into clean, structured datasets.

Learning Objectives

This assignment is built around web scraping specifically. By completing it, you will practice:

- Sending and inspecting HTTP requests, and reading raw HTML with `requests`.
- Inspecting a page's DOM structure using browser developer tools.
- Parsing HTML and extracting structured data with `BeautifulSoup` (CSS selectors, and XPath where useful).
- Automating a real browser with `Selenium`, including explicit waits for elements that load asynchronously.
- Handling pagination, multi-page navigation, and dynamically loaded/scrolled content.
- Chaining a dynamic discovery step (Selenium) with a static extraction step (requests/BeautifulSoup) in the same pipeline.
- Assembling scraped records into a clean, structured dataset with `pandas` and exporting it to CSV.

Important: Focus of This Assignment

This is a **data collection** assignment, not a data-cleaning or exploratory-data-analysis assignment. You are not required to compute statistics, produce visualizations, or train any model. You are expected to

build a working scraper, assemble a clean, well-structured dataset from what you scrape, and briefly document how your scraper works.

Question 1 — Static Web Scraping (10 Marks)

GameVault's storefront lists its entire catalog across a very long sequence of listing pages — more than ninety of them. Unusually, the site will only serve you page 2 onward if your scraper keeps the **same session** it was given on page 1; requesting a later page with a fresh, independent request redirects you back to the start. Every product also has its own dedicated detail page with information not shown on the listing itself.

Target URL: https://sandbox.oxylabs.io/products

Task

1. Starting from the target URL, systematically work through the **entire catalog** by following the site's own pagination, so that no listing page is skipped and none is scraped twice. This is a full-catalog migration, so partial coverage is not acceptable.
2. Maintain a **single, consistent session** (e.g. a `requests.Session()`) across all of your page requests — confirm for yourself what happens if you request page 2 or later as an independent, fresh request instead, and note what you observed in your methodology write-up.
3. For every product you encounter on a listing page, visit that **product's own detail page** to collect the fields that are only available there.
4. Your program must **determine on its own** when it has reached the final page — do not hard-code the total number of pages (there are roughly ninety-plus; the exact number should be discovered by your scraper, not assumed by you).
5. Some products are marked **out of stock** rather than in stock — capture this as its own field rather than ignoring it or treating it the same as an in-stock product.
6. Make sure your final dataset contains **no duplicate products** (a product could otherwise be picked up twice if you re-run parts of your scraper).
7. Given how many requests this catalog requires, decide, and justify in your methodology write-up, how you would make your scraper resilient to an occasional failed request (e.g. a simple retry strategy) so that one hiccup ninety pages in does not throw away everything you had already collected.

Your dataset must include at least the following columns

- Product name
- Price
- Stock status (in stock / out of stock)
- Any additional description text available on the detail page
- The product's own detail-page URL
- The listing page number on which the product first appeared

Save your results as **23L_XXXX_versionA_static_products.csv**.

Question 2 — Dynamic Web Scraping (15 Marks)

StyleHub (separately from the GameVault migration in Question 1) is testing a new landing page for its own catalog that continuously loads more products as the visitor scrolls toward the bottom of the page, instead of using numbered pages. Requesting this URL directly with `requests` will not reveal the later products — a real browser has to actually trigger the loading behaviour by scrolling. Every product tile here is also a link to its own detail page, so — exactly like in Question 1 — this task chains a dynamic discovery step with a static per-product extraction step, this time driven by scrolling instead of paging.

Target URL: https://www.scrapingcourse.com/infinite-scrolling

Task

1. Use **Selenium** to open the page and progressively **scroll** it, using appropriate **explicit waits** for newly loaded product elements to appear, rather than fixed `sleep()` calls alone.
2. Keep scrolling and collecting until scrolling further stops adding new products, and make sure your program can detect this stopping condition itself (for example, by checking whether the number of product elements on the page changed after a scroll).
3. Record, for every product, **which scroll "batch" it first appeared in** (i.e. was it present before you scrolled at all, or did it only appear after your 1st scroll, 2nd scroll, and so on). This is required so you can show the loading behaviour actually worked, not just the final result.
4. Every product tile links to its own detail page (the same kind of page you used in Question 1). For each unique product you discover through scrolling, **follow that link and extract the same detail-page fields you extracted in Question 1** (SKU/identifier and short description at minimum).
5. Be careful not to re-visit the same product's detail page twice if it appears more than once during scrolling (e.g. due to how the page renders while loading).
6. Because the detail pages themselves are static, decide whether to fetch them with `requests` while your Selenium session keeps scrolling, or to navigate to them within the same browser session — justify your choice, including what happens to your scroll position if you navigate away and need to come back.
7. Your final dataset should distinguish products that were on the page **before any scrolling occurred** from products that only appeared **because of** the scrolling behaviour — this is the core proof that your dynamic handling is doing something real.

Your dataset must include at least the following columns

- Product name
- Price
- Image URL
- Scroll batch in which the product first appeared (0 = present before scrolling)
- SKU / unique identifier (from the product's detail page)
- Short description (from the product's detail page)
- The product's own detail-page URL

Save your results as **23L_XXXX_versionA_dynamic_products.csv**.

Scraping Methodology (Required Documentation)

Include a short (roughly half a page to one page per question) write-up, either as Markdown cells in your notebook or a separate short report, briefly explaining:

- How you identified the relevant elements/records on each page (your general approach, not a line-by-line selector dump).
- How you navigated across pages (pagination, scrolling, clicking, etc.).
- How you handled dynamic content in Question 2, including any waits you used and why.
- How your program decided that scraping was complete (i.e., how it knew there were no more pages/records).
- Any challenges you ran into and how you resolved them.
- How you verified that your scraper was actually collecting correct, complete data (spot checks, counts, etc.).

Keep this concise — this should not turn into a long report.

Validation — Scraping Statistics (Required)

For each question, report basic statistics that show your scraper actually worked, for example:

- Total number of pages/scrolls/clicks processed.
- Total number of records collected.
- Number of records successfully extracted with all required fields.
- Number of records skipped, empty, or failed (if any), and why.

This is only meant to confirm your scraper's correctness — it is **not** a data-cleaning or EDA exercise.

Dataset Publication (Required)

Upload the final datasets generated by your two scrapers to either **Kaggle** or **GitHub** (a single dataset repository/notebook containing both CSV files is fine) and provide the public URL in your submission. The published data must be exactly what your own scraper produced — manually constructed or edited datasets will not be accepted.

Bonus — Playwright (+5 Marks)

StyleHub's engineering team is evaluating whether to standardize on Playwright instead of Selenium for future scraping and testing work.

Task

1. Re-implement a meaningful portion of Question 2's dynamic scraping process using **Playwright** instead of Selenium.

2. Your Playwright implementation does not need to reproduce the full dataset — it should demonstrate the same core interaction (waiting for and collecting the dynamically loaded content) correctly.
3. Write a short comparison (a few sentences) of Selenium vs Playwright for this specific task, considering waiting/synchronization, element selection, browser interaction, code complexity, and reliability.
4. Conclude with which tool you would prefer for this particular scraping task, and why, based on your own implementation experience.

The Playwright bonus is optional and is intended to introduce you to a second browser-automation tool — it is not a required part of the core 40 marks.

Submission Requirements

Your submission zip file must contain:

- A Jupyter Notebook (**23L_XXXX.ipynb**) containing both scrapers (and the Playwright bonus if attempted), clearly organized by question.
- Your two final scraped datasets (**23L_XXXX_versionA_static_products.csv** and **23L_XXXX_versionA_dynamic_products.csv**).
- A short methodology write-up (either inside the notebook as Markdown or as a separate PDF/short document) covering the points listed under Scraping Methodology above.
- A text file or Markdown cell containing the public Kaggle or GitHub URL where your dataset(s) are published.

Marking Scheme

Component	Marks
Question 1 — Static Web Scraping	10
Question 2 — Dynamic Web Scraping	15
Dataset Generation (structure, completeness, correctness)	5
Scraping Methodology / Documentation	5
Code Quality / Reproducibility	5
Total (Core Assignment)	**40**

Playwright Bonus: +5 marks (awarded separately, on top of the 40 core marks).